

WEATHER BASED SONG RECOMMENDER - REPORT

Authors: Alia Sabado, Jisoo Im, Puja Shah

CMPT353 - Spring 2025

Professor: Steven Bergner

The Problem

1.1 Introduction

Most music recommendation systems rely on collaborative or content-based filtering to suggest songs based on past user behaviour or song similarity. However, music preferences are often influenced by emotional and environmental factors, particularly the weather, which traditional systems overlook.

This project proposes a context-aware recommendation system that integrates two key data sources: a user's Spotify listening history and corresponding weather data. By analyzing how music choices vary with weather conditions (e.g., genre preferences on rainy vs. sunny days), the system predicts what types of songs a user is likely to enjoy under current conditions. The result is a personalized, dynamic playlist that aligns with both the user's recent listening patterns and the current weather, offering a more emotionally relevant music experience.

1.2 Goals of the Project

The goal of this project is to develop a personalized music recommendation system that integrates a user's Spotify listening history with weather data to generate context-aware playlists. Specifically, we aim to:

- Analyze a user's Spotify listening behaviour over the past year to identify trends.
- Collect historical and real-time weather data (e.g., icon, temperature, cloud coverage) using a weather API.
- Identify correlations between weather features and music preferences (e.g., genre or mood).
- Build a model that recommends songs based on both current weather conditions and recent listening history (within the last 10 days).
- Generate playlists that are not only personalized but also emotionally aligned with the user's current environment.

This system serves as a proof of concept for integrating environmental context into music recommendation algorithms, with potential for future expansion to include factors like time of day, location, or biometric data.

Data: Gathering and Cleaning

2.1 Data Cleaning

We had three different types of data. First is the Spotify listening history, which includes the last 12 months of streaming history. Each row in this dataset corresponds to a song that a user has listened to, where each column contains information on the time that a song ended, the name of the artist, the name of the song, and the number of milliseconds that were played. Next is a Kaggle dataset which includes a song's audio features such as genre, popularity, danceability, energy, etc. Then, finally, we have a weather dataset that contains weather information in Vancouver from March 2024 to March 2025.

We initially attempted to gather song features directly using the Spotify API, but faced two main issues: rate limits and noisy data. Genres were especially problematic, with excessive overlap and inconsistencies (e.g., songs labeled with dozens of highly specific or ambiguous genres). We tried using fuzzy matching techniques like fuzzywuzzy to cluster them into broader groups, but the results were unreliable. Additionally, several critical audio features were sparsely populated or contained NaN values. While exploring alternatives, we discovered a cleaned Kaggle dataset that included most of the desired features. An inner join with our listening history preserved over 70% of our songs and provided consistent, high-quality data, so we chose to proceed with that.

Using this merged dataset, we drop any unnecessary columns, such as track ID, and any rows with missing values. We want to be able to group each song into a cluster. However, since the genre of a song is a categorical variable, we must first convert it to a numerical value using one-hot encoding. The resulting data after conversion has each genre as a column, where the value is either true or false if the song falls under that genre.

For the weather dataset, we selected only the relevant columns for our analysis, which were datetime (for merging), temperature, windspeed, cloud cover percentage, and icon. The icon is a categorical variable, indicating whether it was clear, cloudy, raining, or snowing. Similar to the genres, we convert this numerically using one-hot encoding. Using these two datasets, we merged everything by date to obtain our final dataset, which will be used for most of our modelling.

2.2 Clustering

Using our final dataset, we cluster each of the songs based on their audio features using DBSCAN clustering, a density-based clustering algorithm well-suited for discovering clusters of varying densities and identifying noise (i.e., outliers). However, some clusters were too big. Thus, if a cluster has more than 10% of songs in the entire dataset, we convert it into smaller clusters using K-Means. To avoid confusion between the cluster index labels of DBSCAN and K-Means clusters, we added 1000 to the K-Means cluster ID. After all the clustering is completed, we have a new column that indicates which cluster a song is grouped into.

Analysis: Techniques

3.1 Analysis

Before developing our machine learning model, our group first examined the relationship between music genre and each of the weather features, which were temperature, cloud cover, and icon. We did this in order to see whether the music genre was affected by the features. Using ANOVA for temperature and contingency tables for cloud cover and icon, we obtained an extremely small p-value, leading us to believe that there may be a relationship between genre and weather.

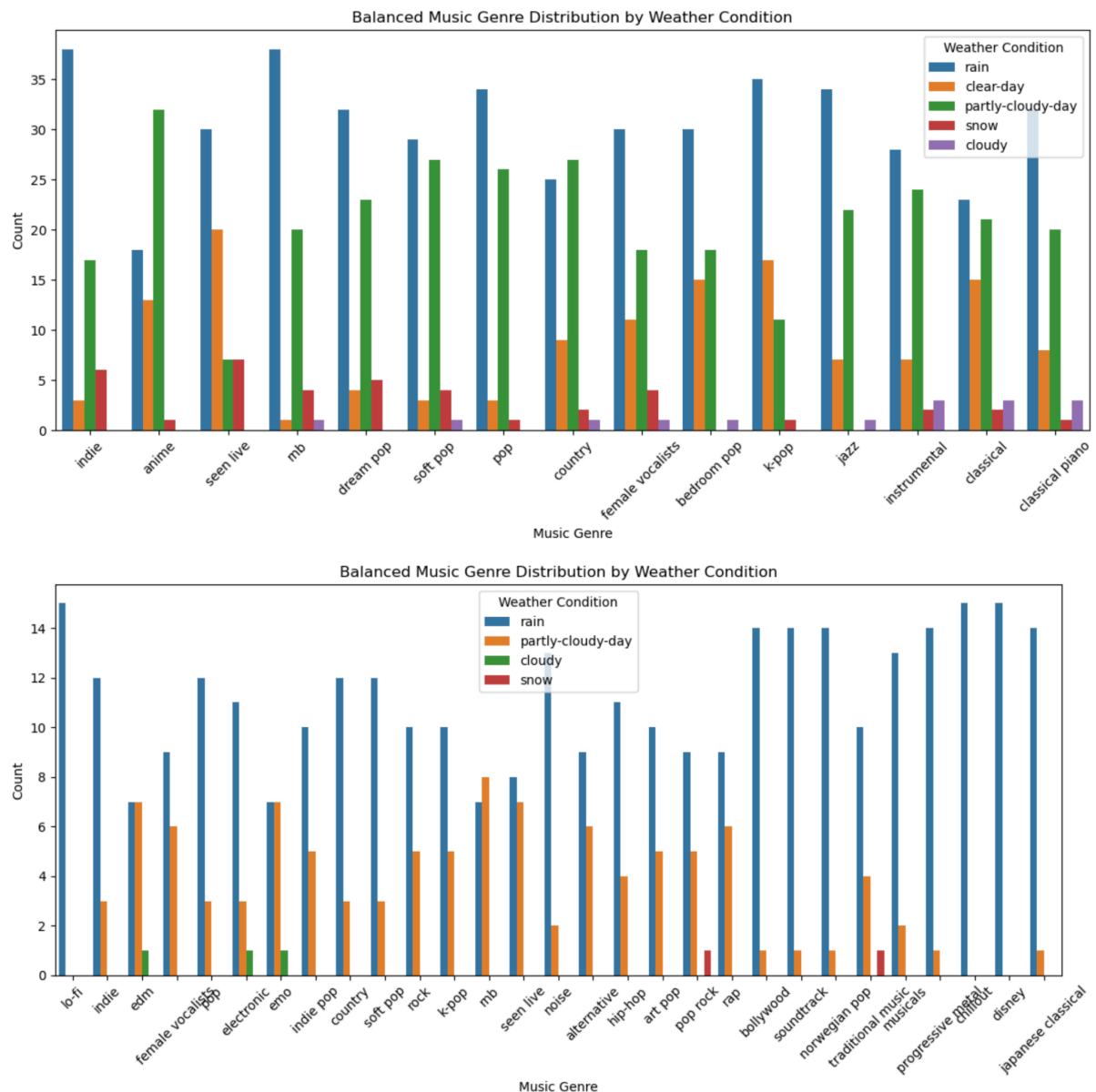


Figure1. Song genre and icon Alia (Top) and Puja (Bottom): This figure displays the genres that appear more frequently than the average count across all genres in the listening history. Only genres with a mean count above the overall average are included.

However, when we tried to fit various machine learning models to predict genres from the weather features, we got quite a low score, which was around 25%. Upon further analysis, we notice that some genres have been listened to over 600 times, while some are as small as 1, making our data severely imbalanced. To solve this issue, we got the average listening counts across all the genres. Using this average, we will sample this many songs from each of the genres. However, we first exclude genres with a listening count that is less than this average, since we cannot sample this number of songs. We also deem these genres to be irrelevant, compared to other genres with a higher listening count. By doing this, we also simplify our data by only including songs that are most listened to, instead of very niche songs that have only been listened to once or twice.

We obtain the count of each genre and join this value to our full data. Then, we remove any rows that are less than our average value. Depending on our two users, this value may be different. After this, we can proceed with sampling random rows for each unique genre in the data and concatenating all these rows together. We end up with a much smaller dataset compared to before, but it is balanced. We do another machine learning model, however, we still get a really low accuracy score.

3.2 Adding more song features and creating the cluster

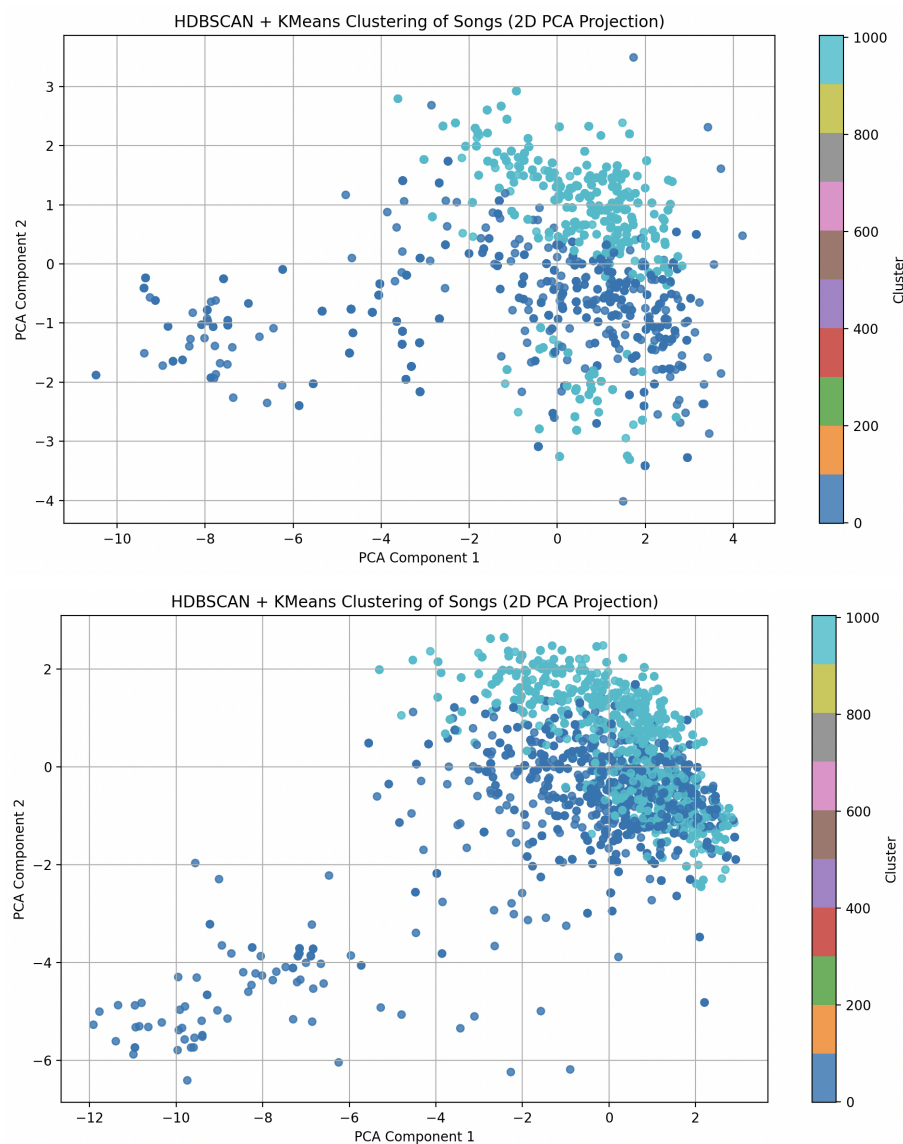


Figure2. Clustering of Songs Alia (Top) and Puja (Bottom). This illustrates the distribution of songs in a 2D PCA space, where each point represents a song and its color indicates cluster membership as determined by a hybrid DBSCAN and K-Means clustering approach.

In the distribution of clustered songs, we applied Principal Component Analysis (PCA) to reduce the high-dimensional feature space (including audio and genre attributes) into two

dimensions. The resulting 2D scatter plot (Figure 2) displays song groupings identified through a combination of DBSCAN and K-Means clustering. Each point represents a song, and the color corresponds to the cluster it belongs to. We observe that several well-defined clusters emerge in the PCA space, indicating that songs with similar characteristics (e.g., energy, acousticness, genre) tend to group together. Some clusters appear more compact, suggesting strong internal cohesion (e.g., tightly grouped upbeat tracks), while others are more spread out or loosely defined. Additionally, a noticeable set of outliers may indicate songs that deviate from typical feature patterns, captured as noise by DBSCAN.

3.3 Model

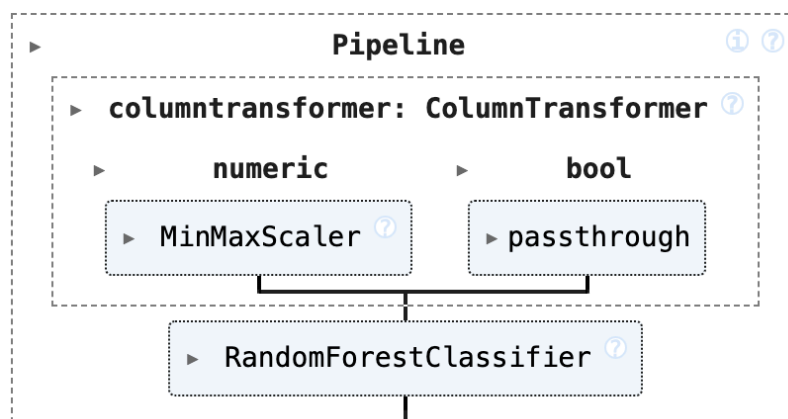


Figure3. The pipeline of the song recommendation model.

In this model, we aim to predict the cluster category of a song based on both weather conditions and audio features. The input data includes key weather indicators such as temperature and cloud cover, along with Spotify-derived song characteristics like danceability, energy, valence, and acousticness. The model uses a Random Forest Classifier trained on these combined features, and preprocessing is handled by a ColumnTransformer that scales numeric values. This setup powers a context-aware music recommendation system that suggests three songs a user is likely to enjoy based on the current weather and patterns learned from their past 10 days of listening history.

Before adding recent listening history, we tried improving predictions by using ANOVA and Spearman correlation to select the top 3 audio features most related to our feature-engineered temp-icon. Despite this, accuracy stayed low. Only after including the last 10 days' averages and keeping all features did scores improve significantly.

3.4 Result

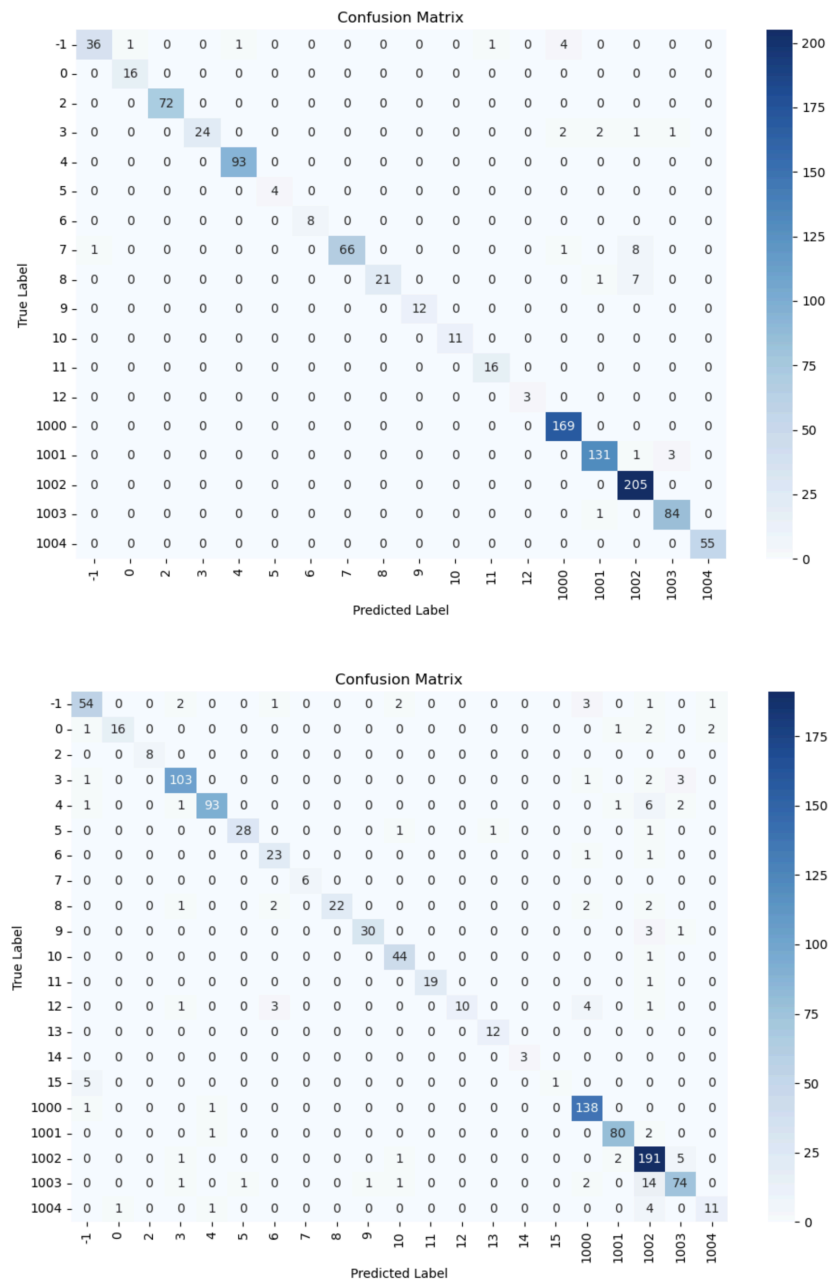


Figure 4. Confusion matrix for Alia (Top) and Puja (Bottom). The confusion matrix visually compares the true cluster labels with the model’s predicted labels.

To evaluate the effectiveness of our context-aware music recommendation model, we trained a Random Forest classifier using both weather data and audio features from historical Spotify listening records. The model achieved an accuracy of over 96% for Alia and 90% for Puja on the validation set, as shown in the confusion matrix (Figure 4). The matrix demonstrates strong diagonal alignment, indicating that most songs were correctly classified into their respective clusters, with very few misclassifications.

To illustrate the model's real-world performance, we tested it with the following weather input.

- Temperature: 12°C
- Cloud Cover: 83%
- Weather Icon: clear-day

Based on this weather context and the user's listening behavior from the past 10 days, the model predicted a cluster and recommended the following three songs. Note that the songs are randomly selected for each predicted cluster.

- Alia: ['unravel', 'Church', 'Into It']
- Puja: ['Monsters (feat. blackbear)', 'Kill Bill', 'Wait a Minute!']

This result demonstrates the model's ability to deliver personalized song suggestions that are not only aligned with the user's past preferences but also dynamically responsive to the current environmental context.

Limitations

Our data is limited to two users, where we have their listening history from February 2024 to February 2025. Because of this limitation, we are only able to analyze the user's listening history from these twelve months. Additionally, the data does not include the location of the user from which they listened to a song. Thus, there is no foolproof way of determining the location of the user, which is relevant for accessing the weather conditions. Therefore, our group is assuming that the two users were located somewhere in Vancouver during the twelve months of their listening history. We used weather data from Vancouver to classify and correlate genres as part of our analysis.

Accomplishment Statements

Project Overview

A music recommendation system that uses machine learning pipelines to suggest songs based on a user's past listening history, genre preferences, and local weather patterns. Songs were clustered using DBSCAN and K-Means based on features such as popularity, danceability, energy, tempo, etc. The clusters were classified using a Random Forest Classifier model to predict a song cluster using weather and song data. The system then recommends three random songs from the predicted cluster to the user.

Individual Contribution (Alia):

- Performed exploratory data analysis on weather temperature to identify trends between daily temperature and user genre preference using plots and ANOVA testing.
- Balanced the dataset by filtering out rare genres and sampling an equal number of songs from the remaining genres to reduce class imbalance.
- Built a machine learning pipeline using ColumnTransformer and MinMaxScaler to preprocess numerical and binary features, scaling numerical features while allowing binary features to pass through.
- Trained a Random Forest Classifier model to predict song clusters using weather features and song characteristics, achieving around 90% accuracy.
- Contributed to the final report, including data cleaning, analysis, clustering, and project limitation sections.

Individual Contribution (Jisoo):

- Collected and processed historical weather data (e.g., temperature, cloud cover, weather icons) to match with timestamped Spotify listening records using a public weather API.
- Discovered correlations between past weather conditions and music genre preferences, enhancing the system's ability to make context-aware recommendations.
- Improved model accuracy and reliability by applying and comparing evaluation techniques, including confusion matrices, accuracy scores, and cross-validation.
- Clearly documented key components of the project in the final report, including introduction, feature analysis, and model performance insights.

Individual Contribution (Puja):

- Used Spotify API calls to gather song features like genre and other audio features and attempted to clean the data using fuzzywuzzy and clustering. Upon finding too many missing values, found the Kaggle dataset and enriched our data
- Performed data cleaning on the song and weather datasets by filtering out unnecessary values, running statistical tests to gauge relationships, and feature engineering columns for future use

- Performed DBSCAN and KMeans clustering on the songs to group them by features. Explored a variety of clustering combinations and parameters and understood the tradeoffs between uniform clustering and accuracy to come to the final model
- Contributed to fine tuning the model by implementing strategies such as reducing audio feature noise and creating ensemble pipelines for models to maximize the score
- Contributed to the final report by enhancing key sections, including data gathering, cleaning, and analysis